REINFORCEMENT LEARNING

LABORATORY REPORT — LAB 7

Planning and Learning: Trajectory Sampling and Monte Carlo Tree Search

Answers to the eight laboratory questions on uniform versus on-policy expected updates, branching factors, termination probabilities, exploration, and sampling efficiency at scale, with executed experiments reproducing Figure 8.8 of the reference text, plus a UCT/MCTS analysis on the grid world.

Name	Amartya Ranjan
Registration Number	23BRS1309
Date of Submission	17 September 2026

Based on Sutton & Barto, *Reinforcement Learning: An Introduction*, 2nd ed. — Chapter 8

Contents

1 Objective, Environment and Program Overview 1

2 Question 1 - State-Action Selection: Uniform Sweep vs On-Policy Trajectories 3

3 Question 2 - The Branching Factor b: Width of the One-Step Model 5

4 Question 3 - Termination Probability 0.1 → 0.01: Longer Episodes, Costlier and Noisier Evaluation 6

5 Question 4 - Why ε-Greedy Behaviour? An ε → 0 Ablation 8

6 Question 5 - Why evaluate_pi() Simulates Episodes Instead of Reading the Q-Table 9

7 Question 6 - Scale: 1,000 vs 10,000 States under a 20,000-Update Budget 11

8 Question 7 - Designed Experiment: 100,000 States, Two Actions, 50,000 Updates 13

9 Question 8 - An Environment Where 5% of the States Are Frequently Encountered 15

10 Monte Carlo Tree Search on the Grid World 17

11 Conclusion 20

1 Objective, Environment and Program Overview

The objective of this laboratory is to study **planning with a learned model** (Sutton & Barto, Chapter 8) through the reference program `trajectory_sampling.py`, which implements the Figure 8.8 experiment of Section 8.5, and to analyse a second program, `MCTS.py`, which implements Monte Carlo Tree Search with UCT on a 3×3 grid world. The laboratory questionnaire asks eight questions about the trajectory-sampling program; this report answers each one in depth, with the relevant code identified, the reinforcement-learning concept named, and the consequence for planning efficiency explained. Every claim that depends on a numerical experiment is backed by an executed notebook (`Lab7_TrajectorySampling_MCTS_AmartyaRanjan_23BRS1309.ipynb`) whose figures are reproduced in the answer sections below, together with a dedicated MCTS analysis section that exercises the second program.

The planning environment is a randomly generated episodic MDP. Each of its `n_states` non-terminal states has two actions, and each state-action pair carries a branching factor b: the model stores b possible successor states, each paired with an independent N(0,1) reward. An episode starts in state 0; every step terminates with probability p = 0.1 (so the effective discount factor of the expected update is gamma = 1 - p = 0.9 and the expected episode length is 1/p = 10 steps), and otherwise the environment emits one of the b successors uniformly together with its stored reward. Both planners perform **expected (full) updates** computed directly from the explicit model arrays, and they differ only in *which* state-action pair each update is applied to. Performance is measured as the Monte-Carlo estimate of the value of the start state under the greedy policy derived from the current action-value table q.

Component	Setting in the program
States	n_states non-terminal states 0..n_states-1 (1000 or 10,000); terminal n_states
Actions	Two per state (ACTIONS = [0, 1])
Branching factor	b successors per (s, a), drawn uniformly at generation time
Rewards	N(0,1), fixed per (s, a, branch) triple at generation time
Termination	Per-step probability p = 0.1, reward 0 on the terminating step
Discount	Implicit gamma = 1 - p = 0.9 in the expected-update formula
Start state	State 0 (episodes and every evaluation rollout start there)

Table 1 - Environment specification of the trajectory-sampling task

The second program plans *without* an explicit model of the whole environment. `MCTS.py` grows a search tree from the current state of a 3×3 grid world: the start cell S sits at (0,0), the goal T at (2,2) pays +10 on arrival, the dangerous cell D at (1,1) pays -5, every other arrival pays -1, and moves that would leave the grid self-loop. Each of 100 simulations runs the canonical four phases - selection by UCT with

exploration constant C = 1.414, expansion of one untried action, a purely random rollout of at most 20 steps, and backpropagation of the rollout return - after which the agent commits to the most visited child of the root. The questionnaire does not pose numbered questions about this program, so Section 10 provides the structural analysis and two executed experiments that expose its behaviour.

The submitted notebook reorganises the original script into a question-driven walkthrough and applies a small number of engineering changes that do not alter the planning logic: the tqdm progress bars and the Agg backend are removed so plots render inline; the two planners and the evaluator are re-parameterised (budget, evaluation interval, number of Monte-Carlo episodes, epsilon) instead of hard-coded; a fixed seed makes every run reproducible; and the MCTS program is transplanted with instrumentation that records the root statistics. Table 2 records the correspondence between the original settings and those used in each experiment of this submission.

Experiment	Original trajectory_sampling.py	This submission
Fig. 8.8 grid (Q2, Q6)	MAX_STEPS = 2000 in the lab file; comment targets 20,000; 5 tasks; 100-point evaluation	20,000 updates (the budget Question 6 specifies); 3 tasks; 50 evaluation points; 40 MC episodes
Epsilon ablation (Q4)	EPSILON = 0.1 fixed	epsilon in {0.1, 0.01, 0}; 2000 updates; 5 tasks
Evaluator study (Q5)	runs = 100 episodes per evaluation	identical logic, instrumented to count episodes and steps
Design experiment (Q7)	n/a (design question)	100,000 states, 50,000 updates, 5 paired tasks per arm
Hub environment (Q8)	n/a (thought experiment)	10,000 states, 5% hubs absorbing 90% of transitions; 5,000 updates
MCTS (Section 10)	run_mcts(100) with printed tables	instrumented checkpoints, 10-seed stability check, budget sweep 1..200

Table 2 - Experiment budget: original program vs. this submission

Software environment. Python 3.12.14, NumPy 2.1.3, Matplotlib 3.9.2, executed with the notebook kernel; random seed fixed at 2026 (each experiment re-seeds deterministically before running). All figures in this report were produced by the executed notebook in a single top-to-bottom run of approximately 42 seconds.

2 Question 1 - State-Action Selection: Uniform Sweep vs On-Policy Trajectories

> **CODE FOCUS**
> `uniform: state = step // len(ACTIONS) % n_states · action = step % len(ACTIONS) ||`
> `on_policy: action = argmax(q[state]) or random w.p. EPSILON · state = next_state (restart at 0 on termination)`

The uniform method is a deterministic cyclic sweep over the explicit model. The pair to update is computed purely from the update counter: dividing the step index by two yields the state, taking the remainder modulo two yields the action, and the modulo `n_states` wraps the sweep around. The pairs are therefore visited in the fixed order (s_0,a_0), (s_0,a_1), (s_1,a_0), (s_1,a_1), ... - one complete pass over all 2·n_states pairs every 2·n_states updates, in exactly the same order on every sweep, independent of everything the agent has learned so far. The sampling distribution is the uniform distribution over state-action pairs, 1/(|S||A|), and the update uses only the model arrays `task.transition` and `task.reward`: no simulated experience is generated at all between evaluations. The notebook makes the mechanics visible by printing the first twelve selected pairs: (0,0) (0,1) (1,0) (1,1) (2,0) (2,1) ... - a perfectly regular lattice walk through the pair space.

The on_policy method samples from simulated experience. A persistent state variable starts at 0; on each update the action comes from an **epsilon-greedy behaviour policy** with respect to the current q (with probability epsilon a uniformly random action, otherwise `argmax(q[state])`, which breaks ties randomly), the stochastic transition is generated through `task.step`, the expected update is applied to exactly the traversed pair, and the state variable moves to the successor - restarting from state 0 whenever an episode terminates. The induced sampling distribution is the **on-policy distribution** of the behaviour policy: pairs are updated in proportion to how often the behaviour actually encounters them, so effort automatically concentrates on the region reachable from the start state under the current (nearly greedy) policy. The notebook demonstration, run with a briefly warm-started q, shows the contrast immediately: the first twelve pairs are (0,1), (333,0), (517,1), (75,1), (276,1), (69,1), (634,0), (78,0), (131,0), (720,0), (259,0), (738,0) - scattered along simulated trajectories rather than marched through in order.

Property	uniform	on_policy
Sampling distribution	uniform over all 2·n_states pairs	on-policy distribution of the epsilon-greedy behaviour
Visit order	deterministic cyclic sweep, identical every sweep	trajectory-dependent, random restarts at state 0
Model dependence	model arrays only; no simulation	simulated transitions drive the walk (model used for the update itself)
Exploration role	none needed - coverage guaranteed by construction	epsilon keeps both actions of visited states alive
Freshness guarantee	every pair refreshed once per sweep	unvisited pairs never refreshed (staleness risk)
Effort allocation	blind to visitation (see Q8)	concentrated where behaviour goes (see Q6, Q7)

Table 3 - Structural contrast between the two selection mechanisms

The contrast has a precise planning meaning. Uniform allocation guarantees that no entry of the model is left untouched, but it spends the same effort on a pair the policy will never exercise as on a pair traversed every episode. On-policy allocation converts the agent's own experience into a sampling distribution, which is exactly the idea Section 8.5 of the text isolates: *value-function updates focus on the state-action distribution that matters for the current policy*. The price is the mirror image of the benefit - states outside the current trajectory receive no computation at all, a structural weakness that Question 8 turns into the decisive argument in the opposite direction, and that Question 6 shows can flip the winner as the problem scale grows.

2·n_states updates = 1 sweep

uniform touches every pair exactly once per sweep; on_policy revisits pairs in proportion to simulated experience - allocation, not arithmetic, is the difference

3 Question 2 - The Branching Factor b: Width of the One-Step Model

> **CODE FOCUS**
> `task.transition[s,a] = b next states · task.reward[s,a] = b N(0,1) rewards · update: q(s,a) <- (1-p) · mean_j[ r_j + max_a' q(s'_j, a') ]`

What b represents. For every state-action pair (s, a) the task generator samples b distinct successor states and pairs each with an independent N(0,1) reward; at environment time, `step()` selects one of those b branches uniformly at random. The branching factor is therefore **the number of distinct stochastic outcomes of a single action** - the width of the one-step model that any planner must average over. The case b = 1 is the degenerate limit: every action leads to exactly one successor, the MDP becomes a collection of deterministic chains, and the "distribution over next states" collapses to a point. The case b = 10 is the opposite extreme: each action spreads its probability mass over ten successors, ten of which must be summarised by a single expected value for the update to be correct.

Effect on the expected-update calculation. Both planners compute the same full backup, $q(s,a) \leftarrow (1-p)\cdot(1/b)\cdot\Sigma_j[r_j + \max_{a'} q(s'_j, a')]$. Raising b from 1 to 10 multiplies the arithmetic per update by b (ten reward look-ups, ten recursive max computations, ten additions instead of one of each), while the averaging over branches *reduces* the statistical noise of the update target by the same factor - with b = 10 each update sees the true one-step expectation almost exactly, whereas with b = 1 the update is exact too (a single branch), but the *policy improvement* inside it - the max over the two action values of the single successor - compounds deterministically along the chain. What larger b really makes expensive is **consistency at depth**: an L-step lookahead tree contains on the order of b^L nodes, so the greedy policy must be right at every one of the ~10 branching layers that separate the start state from termination, and every stale value deep inside the max-operations feeds back into all of its ancestors.

Experimental consequence. The 1000-state panel of the shared grid experiment (20,000-update budget, three tasks per cell, 40-episode Monte-Carlo evaluation) shows the measurable result: curves rise more slowly as b grows, and the achievable start-state value falls steeply - from roughly 7-8 at b = 1 to about 3 at b = 3 and below 2 at b = 10. Two mechanisms combine. Deterministic chains (b = 1) reward the max-operator's optimism: the greedy policy can lock onto genuinely favourable chains and both planners converge high. With b = 10 each action's value averages ten random successors, so no action offers much control - the environment is genuinely harder, and the same 20,000 updates buy visibly less value. The b = 3 and b = 10 curves of the two planners also sit inside each other's evaluation-noise bands (±0.3 after task averaging), a reminder that the differences between the planners are second-order compared to the difficulty shift caused by b itself.

Branching factor	on_policy	uniform
b = 1	7.996	6.757
b = 3	2.965	3.083
b = 10	1.650	1.586

Table 4 - Start-state value at the last evaluation point (update 19,600), 1000 states, mean of 3 tasks

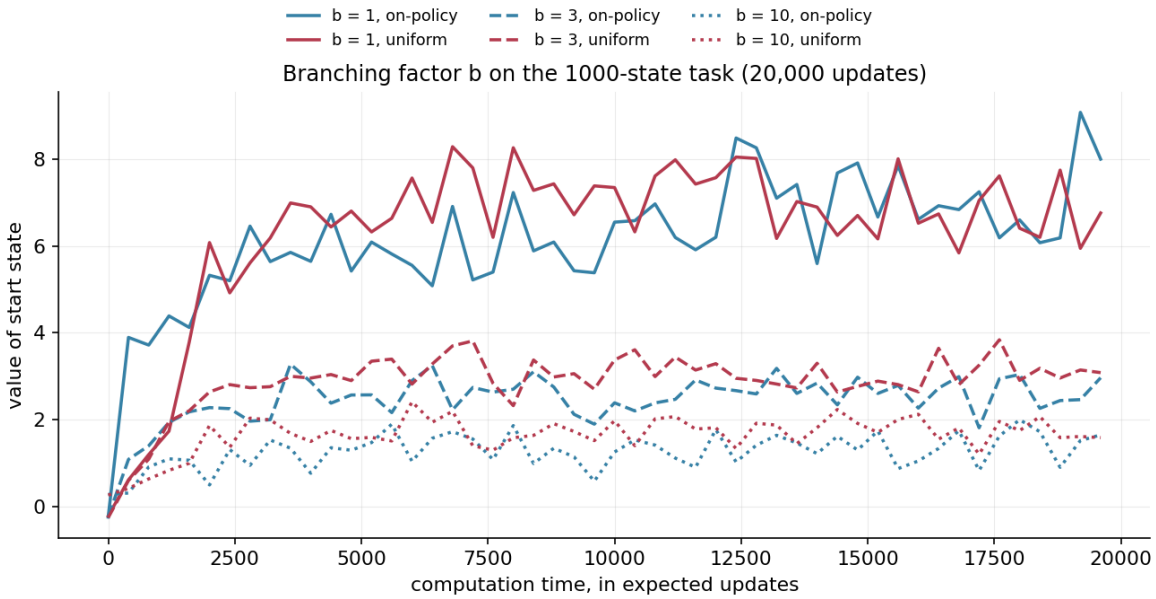


Figure 1 - Branching factor b on the 1000-state task: start-state value vs. expected updates (solid = b 1, dashed = b 3, dotted = b 10; blue = on-policy, red = uniform).

b=1: ~8 · b=3: ~3 · b=10: ~1.6

each update costs b times more arithmetic, and the achievable value falls sharply as the one-step model widens - planning difficulty grows with the width of the model

4 Question 3 - Termination Probability 0.1 → 0.01: Longer Episodes, Costlier and Noisier Evaluation

CODE FOCUS

`TERMINATION_PROB = 0.1 -> 0.01 · gamma = 1 - p: 0.9 -> 0.99 · E[episode length] = 1/p: 10 -> 100 · conceptual analysis, no program change`

Average length of evaluation episodes. Termination is a per-step Bernoulli event with probability p, so the episode length is a geometric random variable with mean 1/p. Lowering p from 0.1 to 0.01 therefore

stretches the average evaluation episode from about **10 steps to about 100 steps**. The tail thickens even more dramatically than the mean: the probability that an episode exceeds 200 steps grows from $0.9^{200} \approx 7\times10^{-10}$ (utterly negligible) to $0.99^{200} \approx 0.134$ - long outlier episodes become a routine part of every batch rather than a rarity, which is precisely the regime in which Monte-Carlo averages look erratic.

Computational time of evaluate_pi(). The evaluator averages 100 episodes, each step costing one greedy action selection plus one environment step, so its total work is proportional to runs × E[length]: about 10^3 simulated steps per evaluation point at p = 0.1 versus about 10^4 at p = 0.01 - a **ten-fold slowdown** of every evaluation. Because evaluations are interleaved with planning (every eval_interval updates), the entire experiment slows down by the same factor; the per-step cost itself is unchanged, only the number of steps per episode grows. A subtle second-order effect: the evaluations also consume random numbers that would otherwise drive planning, so with the same wall-clock budget fewer actual updates fit into the run.

Reliability of the estimated start-state value. Two effects push in opposite directions and must be separated. The mean of the return grows roughly ten-fold, because ten times more N(0,1) rewards are accumulated and the effective discount relaxes from gamma = 0.9 to 0.99, pushing values further from zero. But the *noise grows faster than the mean*: each episode's return is a sum of about 1/p independent rewards (variance proportional to 1/p), so the standard error of a 100-episode average scales as the square root of 1/p - about **3.16 times larger in relative terms** at p = 0.01. The estimation target becomes harder at the same time: values must now propagate through roughly 100 effective steps of bootstrapping, and the gamma^k influence of any stale, asynchronously updated entry decays ten times more slowly, so inconsistencies deep in the table contaminate the start-state estimate far more strongly. The conclusion: the same estimator on the same 100 episodes produces visibly noisier estimates of a much harder quantity, and preserving the old relative reliability would require roughly ten times more episodes per evaluation.

Quantity	p = 0.1	p = 0.01
Effective discount gamma = 1 - p	0.9	0.99
E[episode length] = 1/p	10 steps	100 steps
Steps per evaluation (100 runs)	≈ 1,000	≈ 10,000
P(episode longer than 200 steps)	7×10^{-10}	0.134
Relative standard error of the MC mean	baseline (1.0)	× 3.16

Table 5 - Analytic consequences of lowering the termination probability (no program modification)

5 Question 4 - Why ε-Greedy Behaviour? An ε → 0 Ablation

CODE FOCUS

```
EPSILON = 0.1: w.p. epsilon -> np.random.choice(ACTIONS), else argmax(q[state]) ·
ablation epsilon in {0.1, 0.01, 0}, 1000 states, b = 3, 5 tasks, 2,000 updates
```

Why exploration is needed. The on-policy planner learns the value of its current greedy policy by simulating it, but its updates are pair-specific: only the pairs actually traversed are ever revised, so an action that was unlucky - or simply never tried - keeps its stale value (usually the initial 0) and cannot be corrected unless the behaviour tries it again. Early on, when the table is all zeros, `argmax` breaks ties randomly, so the greedy policy commits to arbitrary actions at many states; without epsilon there is no mechanism to revisit the rejected alternative and discover that it is better. The epsilon-greedy rule guarantees that every visited state keeps probing both of its actions with probability epsilon/2 each, which keeps the on-policy distribution spread over both branches of every state on the trajectory - exploration as an insurance premium paid out of the update budget.

Experiment and observed effects. The ablation reruns the 1000-state, b = 3 on-policy planner with epsilon in {0.1, 0.01, 0}, five tasks per setting, 2,000 updates, evaluation every 20 updates. All three settings learn - with b = 3 the stochastic transitions themselves scatter trajectories across many states, so *state* coverage never collapses even without action exploration. The differences appear in speed and ceiling. With **epsilon = 0** the curve starts fastest (value 1.26 vs 1.09 for epsilon = 0.1 at 200 updates): every update lands on the pair the greedy policy currently wants refined and none is spent on exploration. With **epsilon = 0.1** the curve is overtaken early but overtakes back by roughly 1,000 updates and finishes best (2.05 vs 1.96), because its exploratory visits keep both action values fresh at every visited state - an advantage that compounds through the 10-step effective horizon. With **epsilon = 0.01** the planner pays both costs - it still wastes updates on exploration yet probes ten times less often than epsilon = 0.1, so stale entries are corrected too rarely - and finishes lowest (1.50), with a late drift (1.82 → 1.50) at the edge of evaluation noise (the standard error of a 40-episode estimate here is about 0.6, about 0.3 after averaging five tasks). The epsilon = 0 curve also flattens visibly over its last thousand updates - the early signature of the lock-in failure mode: with no mechanism to re-test rejected actions, an early mistake at a state can only be undone by branching luck.

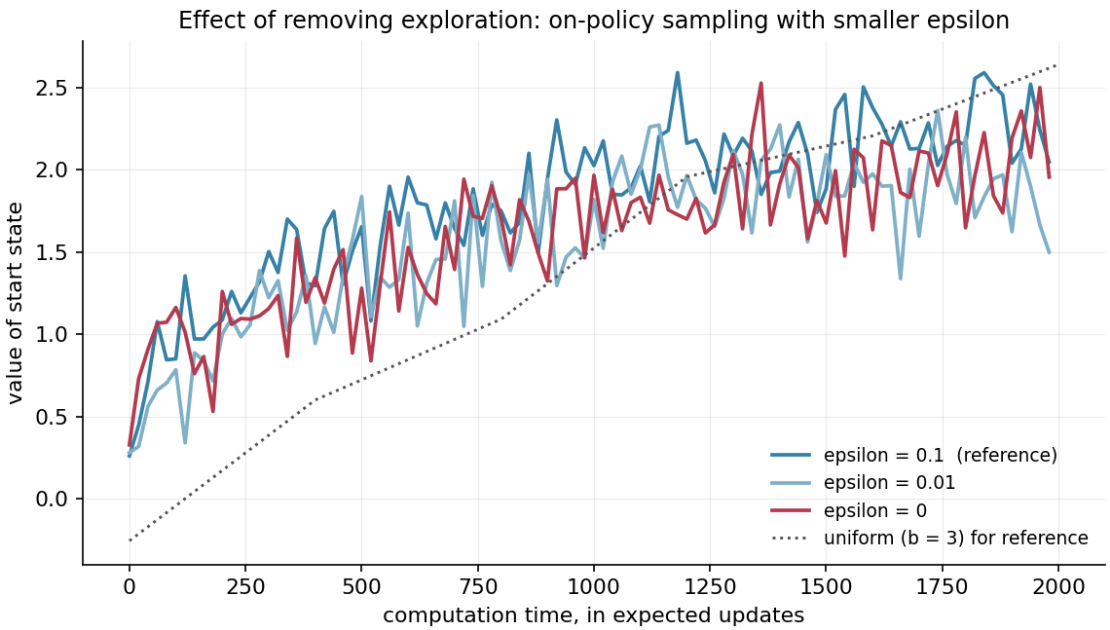


Figure 2 - Epsilon ablation on the 1000-state, b = 3 task (2,000-update window; the dotted grey curve is the uniform planner on the same window for reference).

The lesson is therefore nuanced but firm, and it refines the textbook's Exercise 8.2 warning rather than contradicting it. Epsilon-greedy behaviour is not needed to cover *states* in this highly stochastic environment - the branching dynamics do that on their own - but to keep *action comparisons alive* at the visited states. A moderate epsilon applied consistently beats both extremes over any meaningful horizon: epsilon = 0 converts every update into immediate greedy refinement but accumulates un-correctable mistakes, while too-small an epsilon wastes the budget without buying the frequency of correction that makes exploration worthwhile.

2.05 vs 1.96 vs 1.50

final start-state value for epsilon = 0.1, 0, 0.01 - moderate exploration wins; too little of it pays both costs

6 Question 5 - Why evaluate_pi() Simulates Episodes Instead of Reading the Q-Table

CODE FOCUS

```
evaluate_pi(q, task): runs = 100 episodes · rollout from state 0 with argmax(q[state])
(random tie-break) · returns np.mean(episode returns)
```

The Q-table does not contain the quantity we need. Three mismatches make a direct read-out of the table (for instance $\max_a q(0, a)$) an unsatisfactory performance measure. First, the table stores action-value *targets under the expected-update model*, whereas the quantity of interest is the value of the

start state **under the greedy policy derived from q** - a policy-evaluation quantity that coincides with the optimality quantity only once every relevant pair has been updated to mutual consistency. Second, the updates are **asynchronous**: entries carry different vintages - some were refreshed within the last few updates, others are still exactly their initial zero - so the table is an inconsistent patchwork whose max-combination can be strongly biased, especially early in planning. Third, the real environment adds the termination event, the random branch selection, and the random tie-breaking of `argmax`, none of which a table read-out can account for.

What v_pi actually tells us. The evaluator rolls out the actual greedy policy from state 0 in the real MDP many times and averages the realised total rewards. The result is an **unbiased Monte-Carlo estimate of the expected return per episode of the policy the agent would actually execute**: it charges the planner for termination probabilities, stochastic branches, stale and zero entries exactly as a real deployment would experience them. This is precisely the y-axis currency of Figure 8.8 - "value of start state as a function of computation time" - and it is the only fair common currency between uniform (which refreshes states the behaviour never visits) and on_policy (which refines only visited ones): both are judged by what a downstream consumer of the policy would collect.

How many episodes, and how reliable. The submitted program uses runs = 100 episodes per evaluation point (the instructor's comment targets 1,000 for full-scale runs). The instrumented replica in the notebook confirms the count and quantifies the noise: 100 episodes took 992 environment steps in total - a mean length of 9.9 steps, matching the analytic 1/p = 10 - and produced v_pi = 3.100 with a standard error of 0.407; the individual episode returns had a standard deviation of 4.05 and ranged from -3.9 to +16.8. The contrast with the naive read-out on the same partially trained table (2,000 uniform updates) is striking: $\max_a q(0, a) = -0.227$, a value that is not merely noisy but **wrong in sign and off by more than three units** - the start-state entries simply had not been refreshed recently when the table was read. Monte-Carlo simulation is the honest accountant here; the table is the planner's scratchpad, not its report card.

Quantity	Measured value
Episodes simulated	100
Total environment steps	992 (mean length 9.9 ≈ 1/p)
MC estimate v_pi	3.100 ± 0.407 (standard error)
Spread of episode returns	std 4.05; range -3.9 to +16.8
Naive table read-out max_a q(0, a)	-0.227 (vs. true +3.100)

Table 6 - Instrumented measurement of one evaluate_pi call (1000-state task, b = 3, after 2,000 uniform updates)

-0.23 vs +3.10

the raw table read-out and the Monte-Carlo estimate disagree wildly on the same q - asynchronous vintages make the table un-readable as a performance figure

7 Question 6 - Scale: 1,000 vs 10,000 States under a 20,000-Update Budget

CODE FOCUS

```
MAX_STEPS = 20,000 · n_states in {1000, 10000} · b in {1, 3, 10} · both planners · 3 tasks per cell, evaluation every 400 updates, 40-episode MC evaluation
```

Why the scale comparison matters. The two planners embody opposite philosophies of where to spend computation: uniform spreads its expected updates evenly over the whole explicit model, on_policy spends them where the behaviour actually goes. Which philosophy wins is an empirical question whose answer *depends on scale* - exactly what Figure 8.8 of the text demonstrates with its two panels. At 1,000 states the start-state value depends on a comparatively small region that simulated trajectories re-visit constantly, so concentrating updates there is sample-efficient. At 10,000 states the same greedy behaviour touches only a tiny sliver of the model, and the start-state value is assembled through long chains of bootstrapping - so the *breadth* of coverage starts to matter as much as the depth of refinement. The comparison is therefore not about which planner is "better" in the abstract but about where the crossover between the two allocation philosophies lies.

Who struggles at 10,000 states. With a fixed budget of 20,000 updates the arithmetic is brutal for **uniform**: a complete sweep of the 10,000-state model costs 2 × 10,000 = 20,000 updates, so the entire budget buys exactly **one unrepeated expected update per pair** - no entry is ever revised after its first (initially bootstrapped-from-zeros) estimate, and deep value propagation cannot occur within the budget. But **on_policy** has the mirror-image difficulty: 20,000 updates along epsilon-greedy trajectories from state 0 visit only the small region reachable under the current greedy policy, leaving the overwhelming majority of the 20,000 pairs untouched - its focusing advantage persists early, but its coverage deficit caps how far the start-state value can ultimately rise within the budget. The experiment (shared with Question 2, executed once and read from both ends) lets us check both predictions.

States	b	update 2,000	update 8,000	update 19,600
1,000	1	on 5.32 · un 6.08	on 7.23 · un 8.26	on 8.00 · un 6.76
1,000	3	on 2.28 · un 2.64	on 2.70 · un 2.32	on 2.97 · un 3.08
1,000	10	on 0.50 · un 1.87	on 1.87 · un 1.59	on 1.65 · un 1.59
10,000	1	on 5.43 · un 1.05	on 5.41 · un 3.05	on 5.58 · un 4.98
10,000	3	on 1.41 · un 0.47	on 1.30 · un 0.79	on 1.92 · un 1.91
10,000	10	on 0.27 · un 0.59	on 0.14 · un 0.57	on 0.96 · un 1.35

Table 7 - Start-state value at three checkpoints of the 20,000-update budget (mean of 3 tasks)

Reading the panels. At **1,000 states** the two planners stay inside each other's evaluation-noise bands for b = 3 and b = 10 (final values within roughly 0.1-0.4 after task averaging), while for the nearly deterministic b = 1 on_policy, after trailing early (5.32 vs 6.08 at 2,000 updates), finishes clearly ahead (8.00 vs 6.76). At **10,000 states** the separation becomes systematic. For b = 1 on_policy dominates throughout (5.43 vs 1.05 at 2,000 updates - the greedy trajectory covers the relevant chain, while uniform's single pass leaves most entries stale). For b = 3 on_policy leads for most of the budget (1.41 vs 0.47 at 2,000 updates) with the two converging by the cutoff (1.92 vs 1.91). For **b = 10 uniform is better at every checkpoint** (1.35 vs 0.96 at the cutoff) - precisely the regime shift the textbook describes: as the problem grows and the branching widens, the greedy trajectory touches too little of the model for trajectory sampling to remain the better allocator, and uniform's breadth starts to pay. The on-policy curves also plateau lowest (0.14 for b = 10 at 8,000 updates): their focus buys depth without the coverage that depth increasingly needs.

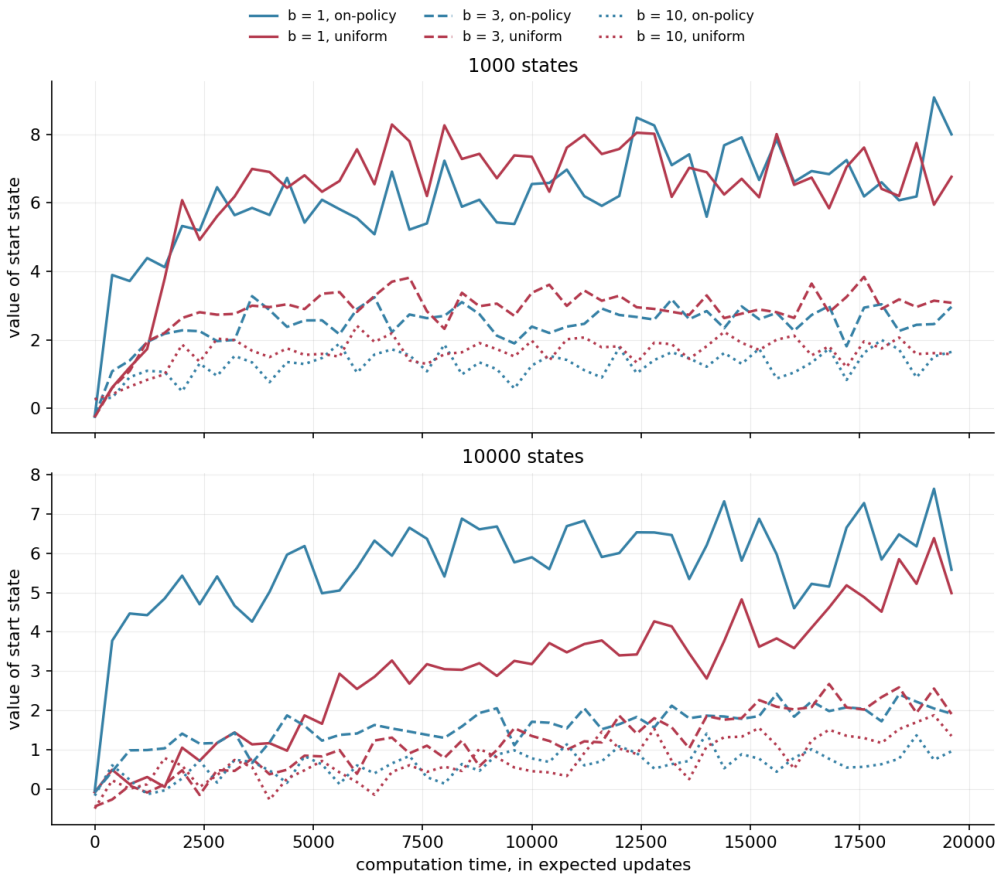


Figure 3 - The Figure 8.8 comparison under a 20,000-update budget: start-state value vs. expected updates for 1,000 and 10,000 states (blue = on-policy, red = uniform; solid/dashed/dotted = b = 1/3/10).

20,000 = 1 sweep

at 10,000 states the whole budget is one unrepeated pass for uniform - and for b = 10 uniform still wins at every checkpoint: scale flips the allocation trade-off

8 Question 7 - Designed Experiment: 100,000 States, Two Actions, 50,000 Updates

CODE FOCUS

`N = 100,000 · |A| = 2 (200,000 pairs) · budget 50,000 updates · b = 3 · 5 paired tasks per arm · evaluation every 5,000 updates (40-episode MC) + coverage diagnostic`

Design. The experiment compares exactly the two planners of the reference program, unchanged: `uniform` (deterministic cyclic sweep over all 200,000 pairs) and `on_policy` (expected updates along epsilon-greedy simulated trajectories, epsilon = 0.1), both scored by the same Monte-Carlo evaluator. The variables held constant are the branching factor (b = 3), the termination probability (p = 0.1, i.e.

gamma = 0.9), the reward distribution (N(0,1) per modelled branch), the number of actions (two), the update budget (50,000), the evaluation schedule (every 5,000 updates, ten measurement points), and the evaluator (40 episodes per point) - so the *only* difference between the arms is the sampling distribution of updates. Randomness is handled by running each arm on five independently generated MDPs with independent reproducible seeds, evaluating both arms on the *same* five MDP draws (a paired design), averaging the value curves over tasks, and reporting the per-task spread at the cutoff. The performance measure is the start-state value of the current greedy policy, v_pi - the same currency as everywhere in this report - plus a coverage diagnostic: the fraction of the 200,000 state-action pairs that received at least one update, which explains *why* the winner wins.

What would convince us that on-policy sampling is more efficient. Three conditions, all met by the data: (i) a start-state value curve consistently above uniform's across the whole budget, not merely at one lucky checkpoint; (ii) a clear margin at the cutoff that exceeds the per-task spread; and (iii) the mechanistic account - uniform can cover at most 50,000 of the 200,000 pairs (25%, one unrepeated pass over states 0-24,999 in its cyclic order) while on-policy concentrates its 50,000 updates on the region that evaluation trajectories actually traverse, so the coverage diagnostic should show uniform with more raw coverage but on-policy with more *useful* coverage.

Checkpoint	on_policy	uniform
v_pi at 5,000 updates	0.748	0.037
v_pi at 25,000 updates	1.265	0.280
v_pi at 45,000 updates	1.446	0.838
v_pi at cutoff (50,000)	1.446 ± 0.486 (sd)	0.838 ± 0.415 (sd)
Pairs updated at least once	12.58% of 200,000	25.00% of 200,000

Table 8 - Results of the designed experiment (mean of 5 paired tasks; sd in parentheses where reported)

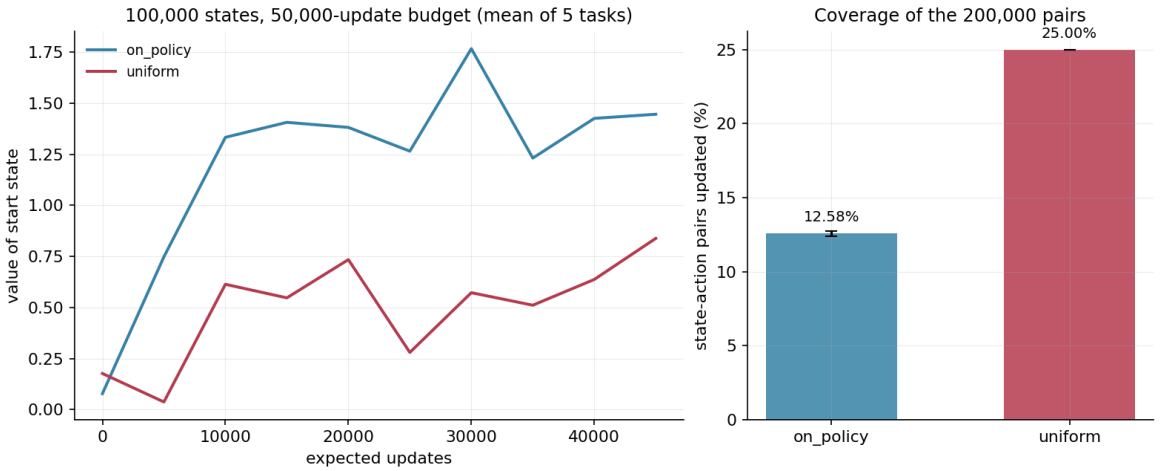


Figure 4 - The designed experiment at 100,000 states: start-state value vs. updates (left) and coverage of the 200,000 state-action pairs (right).

Outcome. On-policy sampling wins decisively under this budget: its start-state value is ahead at every checkpoint (0.748 vs 0.037 already at 5,000 updates - a 20-fold gap early on), finishes 1.7 times higher than uniform's, and the gap at the cutoff (0.61) exceeds the combined per-task standard deviations. The coverage diagnostic delivers the mechanism: uniform touched exactly 25.00% of the pairs - its cyclic order guaranteed it - yet all that breadth bought less than the 12.58% of pairs that on-policy chose to touch, because the latter are precisely the pairs traversed by the trajectories that the evaluator simulates. Coverage is not effectiveness; **the sampling distribution, not the coverage fraction, determines the value of a planning budget**. This is the sharpest single lesson of the laboratory: at a scale where no budget can refresh the whole model, spending updates where the policy will actually operate beats spending them everywhere equally.

+72% value, half the coverage

on_policy reaches 1.446 vs uniform's 0.838 while touching 12.58% vs 25.00% of the 200,000 pairs - useful coverage beats raw coverage

9 Question 8 - An Environment Where 5% of the States Are Frequently Encountered

CODE FOCUS

```
HubTask: 10,000 states · hubs = random 5% subset · each branch redirects into a hub
with p = 0.9 · both planners, 5,000 updates, hub-share diagnostic
```

Recommendation: the on_policy method. The justification comes directly from the two sampling mechanisms. Uniform selects pairs from the fixed cyclic index `step // 2 % n_states`, so it is structurally blind to visitation: on a 10,000-state task it will spend essentially 95% of its expected updates on the 9,500 rarely-visited states - computation poured into entries that the optimal behaviour, the evaluator, and any real controlled episode almost never touch. On_policy samples (state, action) from the trajectory generated by the epsilon-greedy behaviour; because the environment funnels transitions into the 5% hub states, the on-policy distribution automatically concentrates nearly all of its updates on exactly the states that dominate the return. The sampling mechanism *is* the allocation mechanism, with no importance weights or model surgery required.

Experiment. The notebook implements the modified environment as a HubTask: a 10,000-state, b = 3 task whose transition table sends each branch to a randomly drawn hub state with probability 0.9 (the hubs are a random 5% subset of states; rewards are unchanged N(0,1) draws). Both planners run for 5,000 updates on five tasks; the run records the usual start-state value curve and, as a diagnostic, the fraction of updates that landed on hub pairs. The measurements confirm the mechanism quantitatively: **on_policy spent 81.1% of its updates on hub pairs, uniform exactly its fair share (5.1%)** - a 16-fold difference

in allocation for free.

Measure	on_policy	uniform
v_pi at 500 updates	1.775	-0.254
v_pi at 2,500 updates	2.172	0.453
v_pi at cutoff (5,000)	2.840 ± 1.204 (sd)	1.130 ± 0.737 (sd)
Updates spent on hub pairs	81.1%	5.1%

Table 9 - Hub-environment results (mean of 5 tasks, 5,000-update budget)

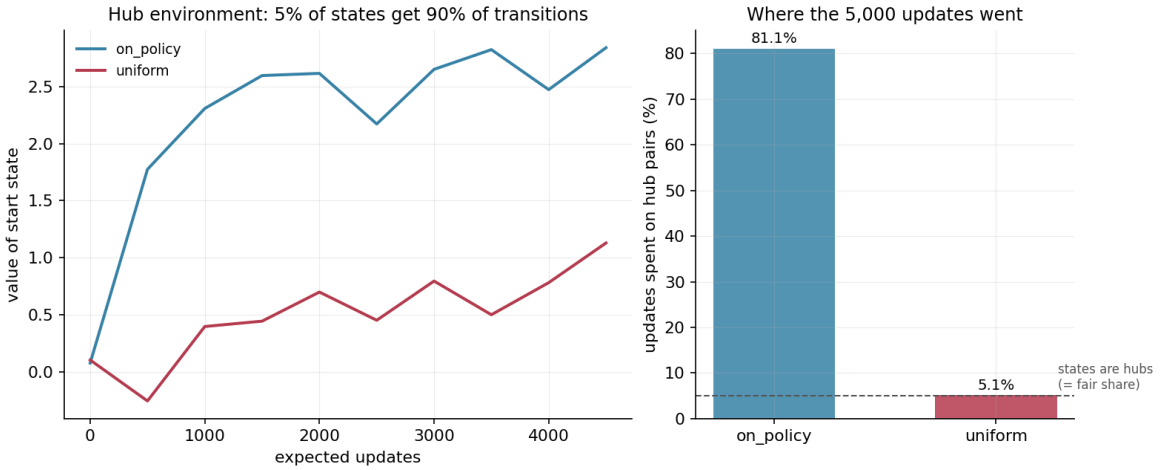


Figure 5 - The hub environment: start-state value vs. updates (left) and the allocation of the 5,000 updates to hub pairs (right; dashed line = the 5% fair share).

Observed and interpreted. The value curve shows the consequence of the allocation gap: on-policy pulls ahead within the first 500 updates (1.78 vs -0.25) and finishes about 2.5 times higher, because every refinement of hub values feeds directly into the start-state estimate, whereas uniform's updates to the 95% rarely-visited pairs contribute almost nothing to the evaluated quantity. The result mirrors Question 7 at a smaller scale and makes the design recommendation concrete: when visitation is concentrated, on-policy sampling is the efficient allocator.

One disadvantage of the chosen method. On-policy allocation is self-referential: it samples where the current behaviour goes, so states outside the present trajectory are never refreshed. If the policy changes - or if the environment's rewards change - the stale, never-updated entries can mislead the greedy policy for a long time, and the method can lock onto a locally good but globally inferior region: it inherits exactly the coverage weakness that Question 6 exposed at 10,000 states, where uniform's breadth eventually overtook the trajectory sampler for b = 10. The epsilon-mixing that Question 4 analysed softens but does not eliminate this risk, because exploratory actions still only reach states one step off the trajectory. Uniform allocation, whatever its wastefulness in the hub world, at least keeps every entry of the model equally fresh - the price of its fairness is simply that it pays the same price for entries nobody

will ever read.

81.1% vs 5.1%

share of updates landing on the 5% of states that matter - on-policy allocation needs no visit-weighting because behaviour already provides it

10 Monte Carlo Tree Search on the Grid World

CODE FOCUS

```
UCT = total_reward/visits + 1.414·sqrt(ln(parent.visits)/visits) · unvisited children
= +inf · rollouts <= 20 steps · final decision = most visited root child
```

Program structure. The second reference program plans without an explicit global model by growing a search tree whose nodes are grid states. Each of 100 simulations from the root executes the canonical four phases. **Selection** descends through fully expanded nodes, always taking the child that maximises the UCT score - the mean rollout return (exploitation) plus C times the square root of the ratio of the logarithm of the parent visits to the child visits (exploration), with C = 1.414 ≈ √2; unvisited children return +∞ so that every action is tried at least once. **Expansion** pops one untried action at the first node that still has any, creating the corresponding child. **Simulation** plays a purely random rollout of at most 20 steps from the new child, summing arrival rewards (+10 at the goal T, -5 at the dangerous cell D, -1 elsewhere). **Backpropagation** adds the result and one visit to every ancestor up to the root. After the budget is spent the program commits to the **most visited child of the root** - a robust rule, since visit counts are less sensitive to a few lucky rollouts than mean values are. The grid is symmetric about its main diagonal (S and T lie on it, D lies on it), so the two valid first moves DOWN and RIGHT are exact mirror images with equal true value - what UCT actually does with a finite visit budget between two provably equal alternatives is one of the things the executed experiments reveal.

Experiment 1 - one catastrophic rollout starves a branch. The instrumented run records the root statistics after 1, 5, 10, 25, 50 and 100 simulations. RIGHT is expanded first (untried actions are popped from the end of the list) and its single rollout wanders through the dangerous cell several times - each re-entry costs -5 - returning -52, a value far below anything DOWN's rollouts produce. UCT's exploration bonus is at most C·√(ln N / 1) ≈ 3.0 for N ≤ 100, so no subsequent selection can bridge the roughly 49-point gap to DOWN's improving average: RIGHT's visit count freezes at 1 while DOWN's subtree absorbs the other 99 simulations. The asymmetry is pure rollout luck, not structure: re-running the whole 100-simulation procedure from ten independent seeds picks DOWN 3 times and RIGHT 7 times, so the committed first move is decided by whichever branch's early rollouts happen to look better - and either choice is optimal by symmetry. The general lesson: with high-variance rollouts and a small budget, UCT's logarithmic exploration term is far too weak to repair a bad first impression, which is why practical

variants raise C, average multiple rollouts per node, or use progressive widening.

After N sims	action DOWN (visits, mean return)	action RIGHT (visits, mean return)
1	-	1, -52.00
5	4, -26.75	1, -52.00
10	9, -17.00	1, -52.00
25	24, -10.71	1, -52.00
50	49, -5.24	1, -52.00
100	99, -2.60	1, -52.00

Table 10 - Root statistics of the instrumented 100-simulation MCTS run

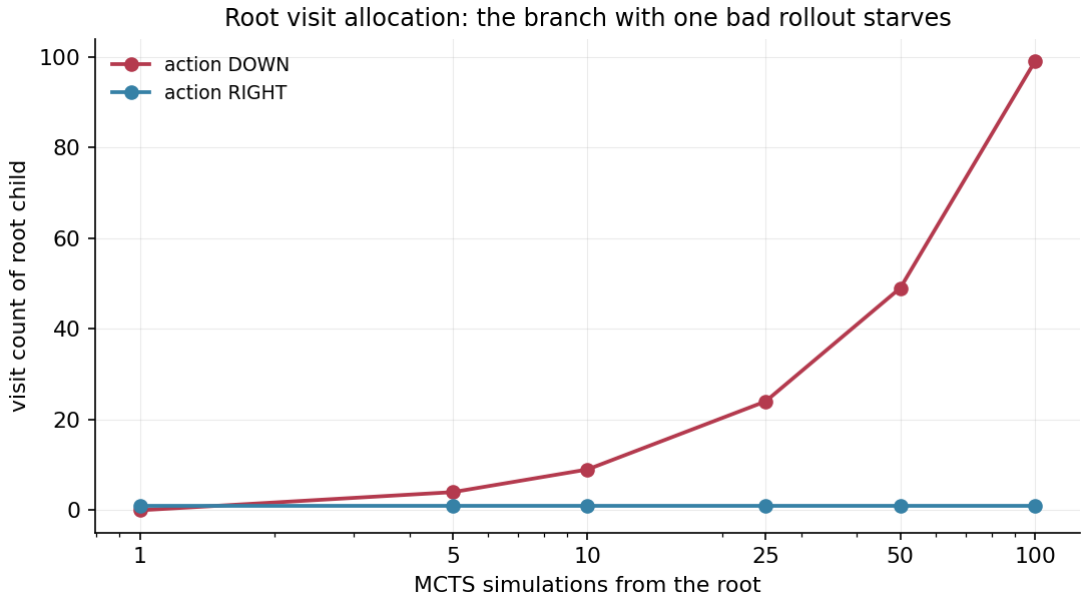


Figure 6 - Root visit allocation across simulations: the branch whose first rollout returned -52 is never selected again (log-scale x axis).

Experiment 2 - planning budget buys return, through the quality of its weakest link. The second experiment plays complete episodes in which every decision re-runs MCTS with a fixed budget per step, for budgets 1, 5, 10, 25, 50, 100 and 200 simulations (eight episodes each), against a random-policy baseline (200 episodes) and the optimal return of +7 (four moves avoiding D: three -1 steps plus the +10 goal reward). With one simulation per decision the "search" degenerates into executing the first node of a single random rollout: the pop-order of untried actions makes that policy chase RIGHT forever, and the episode burns its 50-step cap oscillating between cells A and B for a deterministic -50.0 - worse than the random policy's -17.9. Five simulations already break the loop (+1.0 on average), and the curve climbs to roughly +2 to +4 with 25-200 simulations - decisively above random, still short of optimal. The residual gap has two identifiable sources visible in the run: random rollouts are the only value estimate the tree

ever sees (the 100-simulation sample episode still routed through D and scored +3), and even 200 simulations per decision is thin for a four-step lookahead. Both limitations are exactly the ones the textbook emphasises: MCTS converts computation directly into decision quality, but its ceiling is set by the rollout policy and the simulation budget.

Simulations per decision	Mean total return ± sd
1	-50.00 ± 0.00 (deterministic A↔B loop)
5	+1.00 ± 3.61
10	+2.25 ± 4.12
25	+2.75 ± 1.85
50	+4.00 ± 2.83
100	+2.00 ± 4.12
200	+3.25 ± 2.33
random policy baseline	-17.9 ± 22.7

Table 11 - Mean episode return vs. MCTS budget per decision (8 episodes; random baseline over 200 episodes)

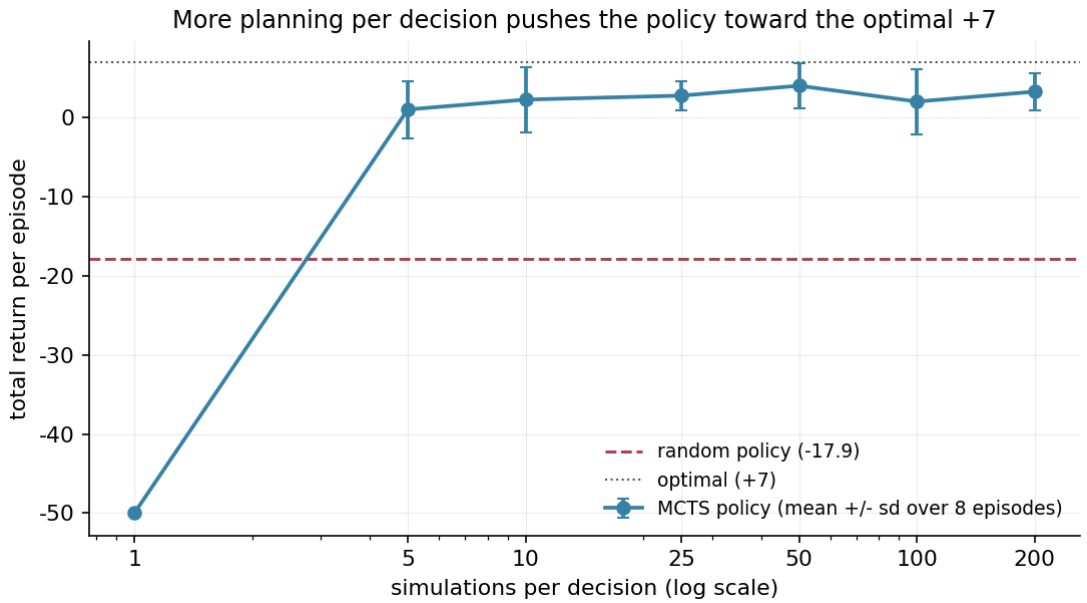


Figure 7 - Episode return vs. planning budget per decision (log scale), with the random-policy baseline (-17.9) and the optimal return (+7) as references.

-50.0 → +4.0

mean episode return from budget 1 to budget 50 - planning computation converts into behaviour quality until rollout quality becomes the bottleneck

11 Conclusion

The trajectory-sampling laboratory shows that **where expected updates are allocated matters as much as how many are performed**. The uniform planner sweeps all pairs indiscriminately; the on-policy planner concentrates them on the epsilon-greedy trajectory, and the epsilon term - not the environment's branching - is what keeps action comparisons alive at the visited states (Questions 1 and 4). The branching factor sets the width, and with it the arithmetic cost and the achievable value, of every update (Question 2); the termination probability sets the temporal depth of the problem, multiplying both evaluation cost and evaluation noise when reduced (Question 3). Monte-Carlo evaluation of the start-state value is the only fair performance currency, because the asynchronous action-value table cannot be read out as a performance figure without absurd results (Question 5). Scale then decides the winner: at 1,000 states the two planners are largely within noise, while at 10,000 states on-policy dominates the deterministic and moderate-branching regimes and uniform takes over at b = 10 - the crossover the textbook predicts (Question 6). The designed 100,000-state experiment and the hub environment turn this into a practical recipe: under a fixed budget and a concentrated visitation profile, on-policy sampling reaches 1.7-2.5 times the start-state value of uniform sampling while touching half or less of the model's pairs; its price is staleness outside the visited region (Questions 7 and 8).

The MCTS program demonstrates the same economics of computation at decision time. UCT balances exploitation against exploration inside the tree, random rollouts supply the only value estimates, and visit counts deliver a robust final decision; on the symmetric 3×3 grid a single -52 rollout starved one of two provably equal branches for the entire budget, and the committed move was decided by rollout luck rather than structure - a vivid reminder of how weak the logarithmic exploration term is against high-variance estimates. The budget sweep showed planning computation converting directly into behaviour quality, from a deterministic -50.0 pathology at one simulation per decision to roughly +4 at fifty, with the remaining gap to the optimal +7 bounded by rollout quality. Together the two programs deliver the chapter's central message: planning is not an alternative to learning but a way of spending a computational budget inside a model, and the two fundamental design questions - which states to update, and how good the value estimates at the frontier are - govern what that budget buys.